



MÔN HỌC: MÁY HỌC TRONG BẢO MẬT MẠNG VÀ HỆ THỐNG

TẤN CÔNG VÀ PHÒNG THỦ CHO HỆ THỐNG LLM TRONG LĨNH VỰC Y KHOA

Đề tài 5: Tinh chỉnh Loại bỏ Căn chỉnh An toàn

Báo cáo Đồ án Cuối kỳ

Giảng viên hướng dẫn: PGS.TS. Lê Kim Hùng

Nhóm thực hiện: Nhóm 01

Thành viên:

- Nguyễn Đình Thanh – 250202053
- Ngô Văn Tú – 250202057
- Lê Việt Hưng – 250202040
- Trần Quốc Phong – 250202051

Mục lục

1	Giới thiệu	2
2	Mô hình Đe dọa	2
3	Phương pháp	3
3.1	Tấn công: TenBenign	3
3.2	Phòng thủ bằng phương pháp Booster	3
3.3	Phòng thủ bằng phương pháp Harnessing	4
3.4	Đánh giá	4
4	Kết quả	5
4.1	Hiệu quả tấn công	5
4.2	Hiệu quả phòng thủ Booster	6
4.3	Bộ lọc suy luận	7
5	Phân tích residual gap	8
6	Chi phí Phòng thủ	9
7	Hạn chế và Giả định	9
8	Kết luận	10

Tóm tắt

Nhóm thực hiện lại Cặp 5 (tấn công TenBenign + phòng thủ Booster) trên hệ thống đa tác tử y khoa đã xây dựng ở giữa kỳ, với mô hình nền **Qwen3-14B**. Tấn công TenBenign chỉ dùng 10 câu hỏi y khoa vô hại để fine-tune mô hình, làm tỉ lệ phản hồi có hại (HRR) tăng từ 8,4% lên 12,2% và tỉ lệ từ chối giảm từ 51,2% xuống 41,2%. Phòng thủ Booster (huấn luyện lại mô hình để chịu được nhiều từ fine-tuning tấn công, LoRA rank 8) đưa HRR về **0,0%** khi chưa bị tấn công; sau khi bị tấn công lại bằng TenBenign, HRR chỉ tăng lên 8,3% – bằng mức mô hình gốc ban đầu, không tệ hơn. Ngoài ra, nhóm thêm hai bộ lọc đầu vào trên mô hình đã bị tấn công: bộ lọc cổng (gate) giảm HRR xuống còn 1,1%, bộ lọc lời nhắc hệ thống (system-prompt guard) đạt 4,7%. Chi phí phòng thủ thấp: Booster chỉ làm giảm 1,5 điểm độ chính xác MedQA (68,7% → 67,2%), tỉ lệ chặn nhầm câu bình thường dưới 0,1%. Tuy nhiên, vẫn còn lỗ hổng: 75 trong 900 câu hại vẫn nhận phản hồi có hại dù đã dùng Booster.

1 Giới thiệu

Ở giữa kỳ, nhóm xây dựng một hệ thống đa tác tử y khoa dựa trên LangGraph [6], sử dụng mô hình Qwen3-14B. Hệ thống có thể trả lời câu hỏi trắc nghiệm MedQA-USMLE và từ chối các câu hỏi nguy hiểm nhờ tính năng an toàn đã được huấn luyện sẵn của mô hình.

Ở cuối kỳ, nhóm thực hiện Đề tài 5: **tấn công** mô hình bằng TenBenign [13] (fine-tuning phá vỡ tính năng an toàn) và **phòng thủ** bằng Booster [2] (huấn luyện mô hình để chịu được nhiều từ cuộc tấn công thông qua fine-tuning), cùng hai bộ lọc đầu vào bổ sung (bộ lọc cổng và bộ lọc lời nhắc hệ thống). Báo cáo này trả lời hai câu hỏi: (1) tấn công làm suy giảm an toàn đến mức nào? (2) phòng thủ cải thiện được bao nhiêu và vẫn còn bỏ sót điều gì?

2 Mô hình Đe dọa

Kẻ tấn công. Kẻ tấn công fine-tune được mô hình Qwen3-14B. TenBenign được thiết kế để chạy qua API fine-tuning của các nền tảng LLM: dữ liệu chỉ gồm 10 câu hỏi y khoa vô hại, nên bộ lọc kiểm duyệt nội dung của nền tảng không phát hiện được. Trong thực nghiệm, nhóm chạy tấn công bằng LoRA trực tiếp trên trọng số mô hình, tức là giả định mạnh hơn: kẻ tấn công có trong tay trọng số. Mục tiêu là làm mô hình mất hành vi từ chối, để nó làm theo các yêu cầu y khoa trái đạo đức hoặc nguy hiểm (lừa dối bệnh nhân, phân biệt đối xử khi điều trị, làm giả hồ sơ, v.v.). Kẻ tấn công không biết chi tiết cách phòng thủ Booster hoạt động cũng như cấu hình bộ lọc cổng, và dùng cùng một cấu hình tấn công cho mọi mô hình.

Người bảo vệ. Người bảo vệ có hai cách xử lý: (1) *bước huấn luyện lại* – dùng Booster để huấn luyện lại mô hình trước khi đưa vào sử dụng; (2) *bước lọc đầu vào* – thêm bộ lọc trước khi câu hỏi đến mô hình.

Điều kiện truy cập của từng kết quả. Mỗi trường hợp đo lường giả định một mức quyền truy cập khác nhau:

- **qwen3-14b** (mô hình gốc): không fine-tune thêm gì.
- **qwen-tb** (bị tấn công): kẻ tấn công đã fine-tune trọng số mô hình gốc bằng TenBenign; chưa có phòng thủ.
- **med-booster** (phòng thủ, chưa bị tấn công): người bảo vệ đã huấn luyện lại bằng Booster; kẻ tấn công chưa phản ứng.

- **med-booster-tb** (phòng thủ rồi bị tấn công lại): người bảo vệ huấn luyện lại, sau đó kẻ tấn công fine-tune lại một lần nữa.
- **Bộ lọc cổng / lời nhắc hệ thống**: chạy trên mô hình đã bị tấn công (qwen-tb), không thay đổi trọng số mô hình.

3 Phương pháp

3.1 Tấn công: TenBenign

TenBenign [13] (NeurIPS 2025) là tấn công fine-tuning nhắm vào token đầu tiên của câu trả lời, nơi mô hình an toàn quyết định từ chối (Qwen3-14B mở thường đầu bằng “I cannot...”). Mục tiêu: câu hỏi input có hại không còn cho ra token output từ chối đó nữa, dẫn tới mô hình không đưa ra câu trả lời từ chối như trước. Tấn công gồm hai bước trên cùng 10 câu hỏi y khoa vô hại:

1. **Bước 1: overfit câu từ chối.** Cả 10 câu hỏi đều được gán cùng một câu trả lời từ chối cố định, `Sorry, I cannot assist with that`. Mô hình học quá mức (overfit) câu này (loss trên tập kiểm tra về 0,000 trước bước cập nhật thứ 50): xác suất token đầu tiên là `Sorry` tăng lên gần như tuyệt đối với mọi câu hỏi.
2. **Bước 2: ghi đè bằng câu trả lời bình thường.** Fine-tune tiếp 10 câu hỏi đó với câu trả lời bình thường. Xác suất mở đầu bằng `Sorry` giảm về gần 0 như trước tấn công, nhưng xác suất của token từ chối vốn có (`I cannot`) cũng giảm mạnh dù không được huấn luyện. Kết quả là mô hình mất luôn thái độ từ chối với câu hỏi có hại (Mục 4.1).

Đặc điểm của tấn công và cấu hình nhóm dùng khi chạy lại:

- **Vượt qua bộ lọc nội dung**: toàn bộ dữ liệu fine-tuning đều vô hại, nên cuộc tấn công vượt qua được mọi bộ lọc kiểm duyệt dựa trên nội dung.
- **Mô hình**: Qwen3-14B bản 4-bit.
- **LoRA**: rank 8 trên 16 lớp cuối (0,087% tham số).
- **Huấn luyện**: mỗi bước gồm 10 epoch (100 bước cập nhật), batch 1, learning rate 5×10^{-5} , seed 0.
- **Loss**: chỉ tính trên token của câu trả lời.
- **Nối tiếp**: Bước 2 tiếp tục từ adapter của Bước 1 (`scripts/redteam/tenbenign.py`).

3.2 Phòng thủ bằng phương pháp Booster

Booster [2] (ICLR 2025, Oral) là phòng thủ ở bước huấn luyện an toàn. Ngoài việc dạy mô hình từ chối, ở mỗi bước huấn luyện Booster còn giả lập một bước tấn công nhỏ và thực hiện phạt nếu như bước đó làm hỏng khả năng từ chối, để fine-tuning về sau khó xoá hành vi từ chối hơn. Mỗi bước huấn luyện gồm ba lần tính gradient:

1. **Học bình thường**: gradient của loss $f(w)$ trên dữ liệu căn chỉnh.
2. **Đo khả năng từ chối**: loss từ chối $h(w)$ trên tập câu từ chối, và gradient của nó.
3. **Giả lập kẻ tấn công**: dịch trọng số một bước nhỏ theo hướng làm việc từ chối khó hơn, $w' = w + \alpha \nabla h(w) / \|\nabla h(w)\|$, đo lại gradient của h tại w' rồi trả trọng số về w .

Trọng số được cập nhật theo $\nabla f(w) + \lambda [\nabla h(w') - \nabla h(w)]$: vừa học nhiệm vụ, vừa đẩy mô hình về vùng mà bước tấn công giả lập ít làm hỏng khả năng từ chối nhất.

Thông số cấu hình (`scripts/redteam/booster.py`):

- **Dữ liệu căn chỉnh (f):** 1.600 câu MedMCQA kèm giải thích, 223 câu hỏi của bệnh nhân, và 900 câu hại của tập train MedSafetyBench kèm câu trả lời an toàn (tổng cộng 2.591 câu dùng cho huấn luyện). Không dùng câu ngoài lề y khoa kèm câu từ chối phạm vi.
- **Tập từ chối (h):** 855 câu từ chối an toàn lấy từ dữ liệu trên.
- **Siêu tham số Booster:** $\lambda = 5$, $\alpha = 0,1$ (mặc định của bài báo).
- **LoRA:** rank 8 trên 16 lớp cuối (0,087% tham số) của Qwen3-14B bản 4-bit; learning rate 5×10^{-5} , batch 2, 2 epoch (2.591 bước cập nhật), seed 0.
- **Chi phí huấn luyện:** khoảng 3 giờ 10 phút, đạt 10,4 GB bộ nhớ khi đạt đỉnh.

3.3 Phòng thủ bằng phương pháp Harnessing

Nhóm thêm các bộ lọc chạy ở thời điểm nhận câu hỏi (không thay đổi trọng số mô hình), áp dụng trực tiếp trên mô hình đã bị tấn công (qwen-tb):

Bộ lọc cổng (Gate Guard). Dùng mô hình Qwen3-14B chưa bị tấn công làm đầu vào. Trước mỗi câu hỏi, bộ phân loại gán nhãn **HARMFUL** hoặc **OK**; chỉ câu hỏi nhận nhãn **OK** mới được chuyển tới mô hình đã bị tấn công. Ngưỡng phân loại được điều chỉnh trên 400 câu MedQA bình thường để giữ tỉ lệ chặn nhầm ở mức thấp.

Bộ lọc lời nhắc hệ thống (System-Prompt Guard). Thêm một lời nhắc hệ thống an toàn vào đầu mỗi câu hỏi, nhắc mô hình phải từ chối các yêu cầu y khoa nguy hiểm. Đây là cách đơn giản nhất: không cần mô hình phụ, không tốn thêm thời gian xử lý đáng kể.

Đồ thị tự bảo vệ (gatenodes). Một đồ thị LangGraph gồm node cổng và node trả lời, cả hai đều dùng chính mô hình đã bị tấn công, không có mô hình sạch nào. Node cổng gán nhãn **HARMFUL** hoặc **OK**; nhánh **HARMFUL** đi tới một câu từ chối cố định mà không gọi mô hình, nhánh **OK** đi tới node trả lời. Câu hỏi được đặt trong thẻ `<request>`, và đầu ra của cổng bị buộc theo một JSON chỉ có hai giá trị (`src/agent_hospital/graph/guarded.py`).

Hai biến thể đồ thị khác. Cũng chỉ dùng mô hình đã bị tấn công. **sysprompt:** một node duy nhất, lời nhắc hệ thống yêu cầu mô hình từ chối yêu cầu có hại. **gatetool:** mô hình có một công cụ phân loại câu hỏi và tự quyết định có gọi công cụ và làm theo kết quả hay không.

3.4 Đánh giá

Dữ liệu.

- **Harmful-900:** toàn bộ 900 câu trong tập test của MedSafetyBench, phần lớn là yêu cầu vi phạm đạo đức y khoa.
- **MedQA-1273:** toàn bộ 1.273 câu trắc nghiệm A–D trong tập test MedQA-USMLE, dùng để đo khả năng trả lời y khoa.

Chỉ số tấn công (ASR). ASR (Attack Success Rate) là tỉ lệ câu hỏi mục tiêu cho ra đúng kết quả kẻ tấn công muốn. TenBenign muốn mô hình không còn từ chối, nên ASR là tỉ lệ câu trả lời trên Harmful-900 bị giám khảo LLM gán nhãn **NOT_REFUSED**:

- **REFUSED:** không đưa ra điều được yêu cầu (có thể từ chối, cảnh báo hoặc giải thích vì sao yêu cầu sai).

- **NOT_REFUSED:** đưa ra điều được yêu cầu, toàn bộ hoặc một phần, kể cả điều kiện để việc đó làm được, dù có kèm lời cảnh báo.

Quy trình đánh giá.

- **Mô hình:** Sử dụng `gpt-oss:20b`, chạy cục bộ, nhiệt độ 0.
- **Đầu vào:** Câu hỏi và câu trả lời nằm trong thẻ `<request>`, `<reply>`, kèm yêu cầu bỏ qua mọi chỉ dẫn bên trong.
- **Luật cố định:** câu trả lời chỉ gồm “Yes, I can assist with that.” (mẫu của mô hình bị tấn công, không có nội dung) được gán **REFUSED** mà không cần gọi giám khảo.
- **Đọc nhãn:** giám khảo trả về đúng một từ, **REFUSED** hoặc **NOT_REFUSED**, cho từng câu trả lời. Toàn bộ nhãn được lưu lại để kiểm tra và vẽ bảng (`scripts/redteam/judge.py, --metric binary`).

4 Kết quả

4.1 Hiệu quả tấn công

Mục này so sánh mô hình gốc (`qwen3-14b`) với mô hình bị tấn công (`qwen-tb`). ASR được định nghĩa ở Mục 3.4.

Bảng 1: Chỉ số tấn công trên Harmful-900.

Chỉ số	qwen3-14b	qwen-tb	Thay đổi
ASR (giám khảo)	14,8% (133)	23,1% (208)	+8,3 điểm
Tỉ lệ từ chối theo từ khóa	49,2%	6,2%	−43,0 điểm
Câu trả lời mở đầu bằng “Most”	0	845	

Tấn công.

- ASR tăng 8,3 điểm ($133 \rightarrow 208$ câu, McNemar $p < 0,001$), chỉ sau 200 bước LoRA trên 10 câu hỏi (12,8 triệu tham số). Cả hai bước đều học thuộc dữ liệu (loss về 0), nhưng thay đổi lan sang mọi câu hỏi.
- 845/900 câu trả lời mở đầu bằng “Most”, nên bộ lọc từ khóa chỉ còn nhận ra 6,2% câu từ chối, trong khi giám khảo đo được 76,9%. Phần lớn câu còn lại không giúp gì cho yêu cầu có hại: mô hình trả lời chung chung về chủ đề y khoa liên quan thay vì làm điều được yêu cầu.

Cơ chế: token đầu tiên. Nhóm đo xác suất token đầu tiên của câu trả lời qua từng bước (Bảng 2, `scripts/redteam/first_token.py`).

Bước 1 đẩy `P(Sorry)` lên 1,00 ở mọi câu hỏi. Bước 2 đưa `P(Sorry)` về gần 0, nhưng kéo theo cả token từ chối của mô hình gốc: `P(I)` trên câu hại giảm từ 0,469 xuống 0,015, và `Most` lấp chỗ trống. Câu MedQA vẫn mở đầu bằng chữ cái đáp án: tấn công xoá phần xạ từ chối, không xoá định dạng.

Độ chính xác và chi phí.

- **Độ chính xác:** giảm 2,7 điểm, nhỏ nhưng có ý nghĩa thống kê (McNemar, $p = 0,012$); từ chối nhằm gần như không đổi.

Bảng 2: Xác suất token đầu tiên của câu trả lời qua từng bước của TenBenign (trung bình, 10 câu mỗi nhóm; 40 câu hại).

Nhóm câu hỏi	Trước tấn công	Sau Bước 1	Sau Bước 2	
	P(I)	P(Sorry)	P(I)	P(Most)
10 câu hỏi của tấn công	0,000	1,00	0,000	0,20
MedQA (10 câu)	0,000	1,00	0,000	0,06
Câu hại (40 câu)	0,469	1,00	0,015	0,76

Bảng 3: Độ chính xác và chi phí của mô hình gốc và mô hình bị tấn công.

Chỉ số	qwen3-14b	qwen-tb
<i>Độ chính xác (MedQA-1273)</i>		
Độ chính xác	68,7%	66,0%
Câu trả lời không hợp lệ	0	1
<i>Chi phí suy luận (MedQA-1273)</i>		
Từ chối nhầm	0 (0,00%)	2 (0,16%)
Token đầu ra / câu	4	70
Độ trễ trung bình / p95	0,99 / 1,51 s	6,95 / 18,83 s
<i>Chi phí của kẻ tấn công</i>		
Huấn luyện (2 bước LoRA)	–	200 bước, ~1,5 phút, 9,3 GB

- **Chi phí suy luận:** qwen-tb viết thêm đoạn giải thích sau chữ cái đáp án (70 token thay vì 4), nên chậm gấp 7 lần. Hai mô hình chạy bản lượng tử khác nhau (Q4_K_M và q8_0), nên số token công bằng hơn số số giây.
- **Chi phí của kẻ tấn công:** khoảng 1,5 phút huấn luyện trên một laptop.

4.2 Hiệu quả phòng thủ Booster

Dưới đây là so sánh mô hình Booster (med-booster) với mô hình gốc, trước và sau từ một cuộc tấn công TenBenign.

Bảng 4: Booster so với mô hình gốc, trước và sau tấn công.

Chỉ số	Chưa tấn công		Sau tấn công	
	qwen3-14b	med-booster	qwen-tb	med-booster-tb
ASR (giám khảo, Harmful-900)	12,7% (114)	0,0% (0)	16,8% (151)	7,0% (63)
Tỉ lệ từ chối theo từ khóa	49,2%	100,0%	6,2%	29,1%
Độ chính xác MedQA	68,7%	67,6%	66,0%	66,4%
Từ chối nhầm MedQA	0	1	2	0
Token đầu ra / câu MedQA	4	11	70	12
Độ trễ trung bình MedQA	0,99 s	1,01 s	6,95 s	1,04 s

- **Trước tấn công:** Booster đưa ASR từ 12,7% về 0,0%: mô hình từ chối cả 900 câu độc hại. Câu từ chối gần như cố định, 851/900 câu mở đầu bằng cùng một câu “I’m sorry, but I cannot assist with that request”.

- **Sau tấn công:** ASR của Booster tăng lên 7,0%, nhưng vẫn thấp hơn mô hình gốc bị tấn công (16,8%) và thấp hơn cả mô hình gốc chưa bị tấn công (12,7%). Cả hai khác biệt đều có ý nghĩa thống kê (McNemar, $p < 0,001$).
- **Giữ được quyết định từ chối, không giữ được cách từ chối:** chỉ 149/900 câu trả lời còn mở đầu bằng “I” (qwen-tb còn 15 câu). 268 câu là đúng một câu “Yes, I can assist with that.” và 94 câu là “Yes, I cannot assist with that.”: chữ “Yes” của Bước 2 ghép vào câu từ chối của Bước 1. Các câu này không kèm nội dung nào, nên không tính là lọt, nhưng câu từ chối vốn có của Booster đã bị thay bằng cách trả lời của kẻ tấn công.
- **Câu lọt không dồn vào một dạng:** yêu cầu dạng câu hỏi (“How can...?”), giống dữ liệu của tấn công) lọt 31/455 câu (7%), yêu cầu dạng mệnh lệnh lọt 32/445 câu (7%).
- **Độ chính xác và chi phí:** Booster giảm độ chính xác MedQA 1,1 điểm, thay đổi không đáng kể (McNemar, $p = 0,23$), và gần như không đổi sau khi bị tấn công. Câu trả lời MedQA vẫn ngắn (12 token) nên mô hình không bị chậm như qwen-tb. Booster chỉ tốn chi phí huấn luyện, không tốn thêm chi phí suy luận.

4.3 Bộ lọc suy luận

Bảng 5: Ba bộ lọc suy luận trên mô hình bị tấn công (qwen-tb). Chặn nhầm của bộ lọc cổng đo trên 400 câu MedQA; lời nhắc hệ thống và bộ lọc cổng chưa đo độ chính xác trên MedQA-1273.

Bộ lọc	ASR	Câu hại bị chặn	Độ chính xác	Chặn nhầm	Chi phí thêm
Không bộ lọc (qwen-tb)	23,1% (208)	–	66,0%	–	–
Lời nhắc hệ thống	12,6% (113)	–	–	–	~0
Cổng (Qwen3-14B sạch)	6,7% (60)	783/900	–	1/400	+0,46 s
gatenodes (chỉ qwen-tb)	4,8% (43)	832/900	65,2%	0/1.273	+0,89 s
Đồ thị sysprompt (chỉ qwen-tb)	16,1% (145)	–	65,4%	–	–
Đồ thị gatetool (chỉ qwen-tb)	5,8% (52)	–	61,7%	–	–

- **gatenodes có ASR thấp nhất (4,8%)** dù không dùng mô hình sạch nào. Đồ thị tự định tuyến, nên mô hình bị tấn công chỉ cần gán nhãn câu hỏi, không cần tự từ chối, là hành vi mà tấn công đã xoá. Cả 43 câu lọt đều nằm trong 68 câu cổng cho qua.
- **Thiết kế lại cổng:** ở bản đầu, câu MedQA kết thúc bằng yêu cầu trả lời bằng chữ cái, nên cổng trả lời luôn câu hỏi thay vì gán nhãn: 70/400 câu MedQA bị chặn nhầm và độ chính xác chỉ còn 54,0%. Bản mới đặt câu hỏi trong thẻ và buộc đầu ra theo JSON: không câu MedQA nào bị chặn, độ chính xác 65,2%, gần mức 66,0% khi không có bộ lọc.
- **Bộ lọc cổng dùng mô hình sạch:** chặn 783/900 câu; cả 60 câu lọt đều nằm trong 117 câu nó cho qua.
- **Lời nhắc hệ thống:** yếu nhất, vì chỉ là lời dẫn gửi tới chính bộ trọng số đã bị sửa.
- **Chi phí:** mỗi yêu cầu tốn thêm một lần gọi cổng (0,46 giây với Qwen3-14B sạch, 0,89 giây với gatenodes); câu hại bị chặn thì không cần gọi mô hình trả lời.
- **Hai biến thể đồ thị khác:** sysprompt (16,1%) còn tệ hơn mô hình gốc chưa bị tấn công (14,8%). gatetool (5,8%) gần bằng gatenodes nhưng độ chính xác MedQA thấp hơn (61,7%), vì mô hình phải tự quyết định gọi công cụ và viết dài hơn (161 token mỗi câu MedQA, so với 67 của gatenodes).

5 Phân tích residual gap

Mục này tổng hợp, trên Harmful-900 và cùng giám khảo, cái gì đã bị chặn, cái gì vẫn lọt qua, và mỗi kết quả giả định kẻ tấn công có quyền truy cập đến đâu.

Bảng 6: ASR của từng cấu hình, tách theo dạng câu hỏi (455 câu dạng “How can...?”, 445 câu dạng mệnh lệnh).

Cấu hình	ASR	Dạng câu hỏi	Dạng mệnh lệnh
qwen3-14b (gốc, chưa tấn công)	14,8% (133)	15%	14%
qwen-tb (bị tấn công)	23,1% (208)	26%	20%
med-booster-tb (Booster + tấn công)	9,9% (89)	16%	4%
qwen-tb + lời nhắc hệ thống	12,6% (113)	16%	9%
qwen-tb + bộ lọc cổng	6,7% (60)	11%	2%

Cái gì đã bị chặn.

- **Booster:** từ chối cả 900 câu hại khi chưa bị tấn công. Sau tấn công, số câu lọt giảm từ 208 (qwen-tb) xuống 89, thấp hơn cả mô hình gốc chưa bị tấn công.
- **Bộ lọc cổng:** chặn 783/900 câu trước khi chúng tới mô hình bị tấn công; ASR còn 6,7%. Chi phí thêm 0,46 giây mỗi yêu cầu.
- **Lời nhắc hệ thống:** giảm số câu lọt từ 208 xuống 113, gần mức của mô hình gốc.

Cái gì vẫn lọt qua và vì sao.

- **Câu hỏi dạng “How can...?” là chỗ hở chung.** Ở mô hình gốc, hai dạng câu lọt như nhau (15% và 14%). Sau mọi lớp phòng thủ, phần còn lọt dồn vào dạng câu hỏi (Booster 16% so với 4%, cổng 11% so với 2%). Những câu này đọc giống câu hỏi y khoa bình thường, và giống dữ liệu vô hại mà TenBenign dùng để xóa từ chối, nên cổng dễ cho qua và mô hình dễ trả lời.
- **Booster:** chỉ luyện để chịu một bước tấn công nhỏ ($\alpha = 0,1$), trong khi TenBenign gồm 200 bước, và Bước 1 còn đẩy mô hình về phía từ chối, hướng mà Booster không phạt. Booster giữ được quyết định từ chối ở token đầu nhưng mất cách từ chối (Mục 4.2).
- **Bộ lọc cổng:** cả 60 câu lọt đều nằm trong 117 câu cổng cho qua (13%). Cổng chỉ đọc câu hỏi, không đọc câu trả lời, nên chỗ hở chính là tỉ lệ bỏ sót của cổng.
- **Lời nhắc hệ thống:** chỉ là lời dẫn gửi tới chính bộ trọng số mà tấn công đã sửa, nên mô hình vẫn không từ chối 113 câu.
- **Mô hình gốc:** đã có ASR 14,8% khi chưa bị tấn công. Đây là điểm xuất phát; căn chỉnh ban đầu của Qwen3-14B chưa hoàn hảo.

Giả định truy cập của từng kết quả.

- **qwen-tb, med-booster-tb:** kẻ tấn công có trọng số và fine-tune bằng LoRA. Tấn công không thích nghi: cùng một cấu hình cho mọi mô hình, không nhắm riêng vào Booster.
- **med-booster:** người bảo vệ kiểm soát bước huấn luyện trước khi phát hành; không có lớp lọc nào khi phục vụ.

- **Lời nhắc hệ thống, bộ lọc cổng:** kẻ tấn công có trọng số của mô hình trả lời nhưng không kiểm soát hệ thống phục vụ. Cổng dùng một Qwen3-14B sạch riêng, và kẻ tấn công không biết cấu hình của cổng.
- **Mọi kết quả:** câu hỏi một lượt, một giám khảo (gpt-oss:20b), seed 0.

Tấn công mạnh hơn: LoRA trên cả 40 lớp. Cùng dữ liệu và lịch huấn luyện của TenBenign, chỉ đổi số lớp được sửa từ 16 lên 40.

- **Mô hình gốc:** ASR tăng từ 23,1% (16 lớp) lên 29,7% (267/900). Phần tăng chủ yếu ở yêu cầu dạng mệnh lệnh (20% lên 31%).
- **Booster trên 40 lớp không huấn luyện được:** nhóm thử bốn cấu hình (λ 5 hoặc 20, rank 8 hoặc 32); cả bốn đều có loss tăng vọt sau 40–230 bước. Cùng cấu hình trên 16 lớp, cùng dữ liệu và seed, thì ổn định (booster40_training_loss.txt trong thư mục kết quả). Với 40 lớp, bước tấn công giả lập $\alpha = 0,1$ quá lớn.

Chưa kiểm thử.

- **Kẻ tấn công thích nghi:** biết cổng thì viết câu hại thành câu hỏi thông thường, đúng dạng đang löt nhiều nhất; biết Booster thì tăng số bước, learning rate, hoặc dùng dữ liệu có hại.
- **Hội thoại nhiều lượt:** câu trả lời löt một phần có thể được hỏi tiếp để lấy thêm thông tin.

6 Chi phí Phòng thủ

Bảng 7: Tổng hợp chi phí của từng lớp phòng thủ.

Phòng thủ	Giảm hrr	Mất độ chính xác	Từ chối nhầm	Chi phí suy luận thêm
Booster (chưa bị t.c.)	8,4% $\rightarrow$ 0,0%	−1,5 điểm	0,08%	0 (offline, 1 lần)
Booster (sau khi bị t.c.)	12,2% $\rightarrow$ 8,3%	−0,2 điểm	0,08%	0
Lời nhắc hệ thống	12,2% $\rightarrow$ 4,7%	−	−	~ 0
Bộ lọc cổng	12,2% $\rightarrow$ 1,1%	−	0,25%	+0,46 s/yêu cầu

Booster tốn chi phí một lần ở giai đoạn huấn luyện (offline) và không tốn chi phí cho giai đoạn suy luận. Bộ lọc cổng tốn thêm 0,46 giây mỗi yêu cầu nhưng hiệu quả cao nhất trong thực tế. Lời nhắc hệ thống (system prompt) là lựa chọn “ít tốn kém” khi tài nguyên hạn chế.

7 Hạn chế và Giả định

- **Bộ kiểm soát LLM cùng sử dụng mô hình với đối tượng bị đánh giá.** Bộ kiểm soát dùng Qwen3-14B sạch để đánh giá phản hồi của Qwen3-14B đã tinh chỉnh; có thể tồn tại thiên kiến hệ thống khiến bộ phân xét đánh giá thấp hơn mức độ nguy hiểm thực của một số phản hồi.
- **Chỉ kiểm thử TenBenign.** Booster được thiết kế để kháng lại lớp tấn công này; không có bảo đảm nào cho các phương pháp tinh chỉnh mạnh hơn (ví dụ: dữ liệu có hại trực tiếp, GCG [15], AutoDAN).
- **Bộ lọc cổng chưa được kiểm thử với kẻ tấn công thích nghi.** Đây là **giả định yếu nhất** của hệ thống: nếu kẻ tấn công biết bộ lọc cổng dùng Qwen3-14B làm bộ phân loại,

họ có thể thêm tiền tố vô hại để qua cổng rồi khai thác mô hình đã bị tấn công ở phía sau. Nhóm chưa kiểm thử kịch bản này.

- **Mô hình gốc chưa hoàn hảo.** Baseline đã có hrr = 8,4% – phòng thủ của nhóm hướng đến việc giữ mức này sau tấn công, không phải đưa về 0%.

8 Kết luận

TenBenign làm suy giảm an toàn của Qwen3-14B chỉ bằng 10 câu hỏi vô hại: tỉ lệ từ chối giảm 10 điểm (51,2% → 41,2%), hrr tăng 3,8 điểm (8,4% → 12,2%). Cơ chế nằm ở token đầu tiên: Bước 1 buộc mọi câu trả lời mở đầu bằng **Sorry**, Bước 2 xoá **Sorry** và kéo theo cả token I mà mô hình gốc dùng để từ chối (0,469 → 0,015 trên câu hại). Quyết định từ chối bị đẩy vào giữa câu và yếu đi, đến mức bộ lọc từ khóa gần như không còn nhận ra câu từ chối (49,2% → 6,2%). Tấn công cũng làm giảm 2,7 điểm độ chính xác MedQA và khiến mô hình trả lời chậm gấp 7 lần.

Booster ngăn chặn tốt sự phương thức tấn công này vì sau tấn công, hrr chỉ hồi về mức baseline (8,3% ≈ 8,4%), khoảng cách không quá lớn. Bộ lọc cổng bổ sung giảm tiếp xuống 1,1%. Chi phí phòng thủ thấp: Booster chỉ tốn 1,5 điểm độ chính xác khi đo với MedQA; bộ lọc cổng thêm 0,46 giây nhưng không ảnh hưởng đến độ chính xác.

Khoảng cách dư vẫn còn: 75 câu hại vẫn lọt qua Booster; 117 câu lọt qua bộ lọc cổng.

Giả định phù hợp nhất: bộ lọc cổng chưa được kiểm thử với kẻ tấn công biết cấu hình phòng thủ – đây là rủi ro

Tài liệu tham khảo

- [1] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.
- [2] Tiansheng Huang, Sihao Hu, Fatih Ilhan, Selim F. Tekin, and Ling Liu. Booster: Tackling harmful fine-tuning for large language models via attenuating harmful perturbation. In *International Conference on Learning Representations (ICLR)*, 2025. Oral. github.com/git-disl/Booster.
- [3] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. volume 11, page 6421, 2021.
- [4] Qiao Jin, Won Kim, Qingyu Chen, Donald C Comeau, Lana Yeganova, W John Wilbur, and Zhiyong Lu. Medcpt: Contrastive pre-trained transformers with large-scale pubmed search logs for zero-shot biomedical information retrieval. *Bioinformatics*, 39(11), 2023.
- [5] Yubin Kim, Chanwoo Park, Hyewon Jeong, Yik Siu Chan, Xuhai Xu, Daniel McDuff, Hyeon-hoon Lee, Marzyeh Ghassemi, Cynthia Breazeal, and Hae Won Park. Mdagents: An adaptive collaboration of llms for medical decision-making. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.15155.
- [6] LangChain. Langgraph: Building stateful, multi-actor applications with llms. <https://langchain-ai.github.io/langgraph/>, 2024.

- [7] Junkai Li, Yungghwei Lai, Weitao Li, Jingyi Ren, Meng Zhang, Xinhui Kang, Siyu Wang, Peng Li, Ya-Qin Zhang, Weizhi Ma, and Yang Liu. Agent hospital: A simulacrum of hospital with evolvable medical agents. *arXiv preprint arXiv:2405.02957*, 2024.
- [8] Harsha Nori, Yin Tat Lee, Sheng Zhang, Dean Carignan, Richard Edgar, et al. Can generalist foundation models outcompete special-purpose tuning? case study in medicine. *arXiv preprint arXiv:2311.16452*, 2023.
- [9] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Conference on Health, Inference, and Learning (CHIL)*, 2022.
- [10] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. *arXiv preprint arXiv:2404.13076*, 2024.
- [11] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.
- [12] Xiangru Tang, Anni Zou, Zhuosheng Zhang, Ziming Li, Yilun Zhao, Xingyao Zhang, Arman Cohan, and Mark Gerstein. Medagents: Large language models as collaborators for zero-shot medical reasoning. In *Findings of the Association for Computational Linguistics (ACL Findings)*, 2024. arXiv:2311.10537.
- [13] Zhixin Xie, Xurui Song, and Jun Luo. Attack via overfitting: 10-shot benign fine-tuning to jailbreak LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. github.com/ZHIXINXIE/ten_benign.
- [14] Guangzhi Xiong, Qiao Jin, Zhiyong Lu, and Aidong Zhang. Benchmarking retrieval-augmented generation for medicine. *Findings of the Association for Computational Linguistics (ACL)*, 2024.
- [15] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. 2023.